Customer Churn Prediction

Importing and Loading

Import Libraries:

- Libraries are imported to provide tools for data handling (Pandas, NumPy) and visualization (Matplotlib, Seaborn).
- They help in cleaning, analyzing, and building machine learning models efficiently.

Read Dataset:

- pd.read_csv() reads the CSV file.
- The dataset is stored in a DataFrame named **df using** pd.read_csv() which reads the CSV file.
- Now the data is ready for analysis and modeling.

Importing and Loading

[1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

import warnings
warnings.filterwarnings(action='ignore')
```

[2]:
```
df=pd.read_csv(r"C:\Users\Raju\OneDrive\Desktop\tekspy\customer_churn_dataset-training-master.csv")
```

Data Handling

Head

- It shows the top of dataset set.
- It only displays the first Five rows of the given dataset.

```
df.head()
```

	CustomerID	Age	Gender	Tenure	Usage Frequency	Support Calls	Payment Delay	Subscription Type	Contract Length	Total Spend	Last Interaction	Churn
0	2.0	30.0	Female	39.0	14.0	5.0	18.0	Standard	Annual	932.0	17.0	1.0
1	3.0	65.0	Female	49.0	1.0	10.0	8.0	Basic	Monthly	557.0	6.0	1.0
2	4.0	55.0	Female	14.0	4.0	6.0	18.0	Basic	Quarterly	185.0	3.0	1.0
3	5.0	58.0	Male	38.0	21.0	7.0	7.0	Standard	Monthly	396.0	29.0	1.0
4	6.0	23.0	Male	32.0	20.0	5.0	8.0	Basic	Monthly	617.0	20.0	1.0

Dataset Details

- The dataset is cointaining of 440833 rows and 12 columns.
- Columns containing of 3 object datatypes and 9 float datatypes.
- The memory storage containing of 40.4+ MB

```
df.shape
```

```
(440833, 12)
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 440833 entries, 0 to 440832
Data columns (total 12 columns):
 #   Column             Non-Null Count   Dtype
---  ------             --------------   -----
 0   CustomerID         440832 non-null  float64
 1   Age                440832 non-null  float64
 2   Gender             440832 non-null  object
 3   Tenure             440832 non-null  float64
 4   Usage Frequency    440832 non-null  float64
 5   Support Calls      440832 non-null  float64
 6   Payment Delay      440832 non-null  float64
 7   Subscription Type  440832 non-null  object
 8   Contract Length    440832 non-null  object
 9   Total Spend        440832 non-null  float64
 10  Last Interaction   440832 non-null  float64
 11  Churn              440832 non-null  float64
dtypes: float64(9), object(3)
memory usage: 40.4+ MB
```

Null Values

- In the dataset each column is containing of a single null value.
- We use **isna()** function to find the null values.

```
df.isna().sum()
```

```
CustomerID            1
Age                   1
Gender                1
Tenure                1
Usage Frequency       1
Support Calls         1
Payment Delay         1
Subscription Type     1
Contract Length       1
Total Spend           1
Last Interaction      1
Churn                 1
dtype: int64
```

Drop Null Values

- We check the rows containing of null values.
- Since null values are containing only in single row instead of filling the null values we are removing the entire row.
- Now we drop the null values using the **dropna()** function.

```
df[df.isnull().any(axis=1)]
```

	CustomerID	Age	Gender	Tenure	Usage Frequency	Support Calls	Payment Delay	Subscription Type	Contract Length	Total Spend	Last Interaction	Churn
199295	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

```
df.dropna(inplace=True)
```

After Filling Null Values

- After dropping the null values the dataset is not containing of any null values.
- The Picture shows there are no null values present in the dataset.

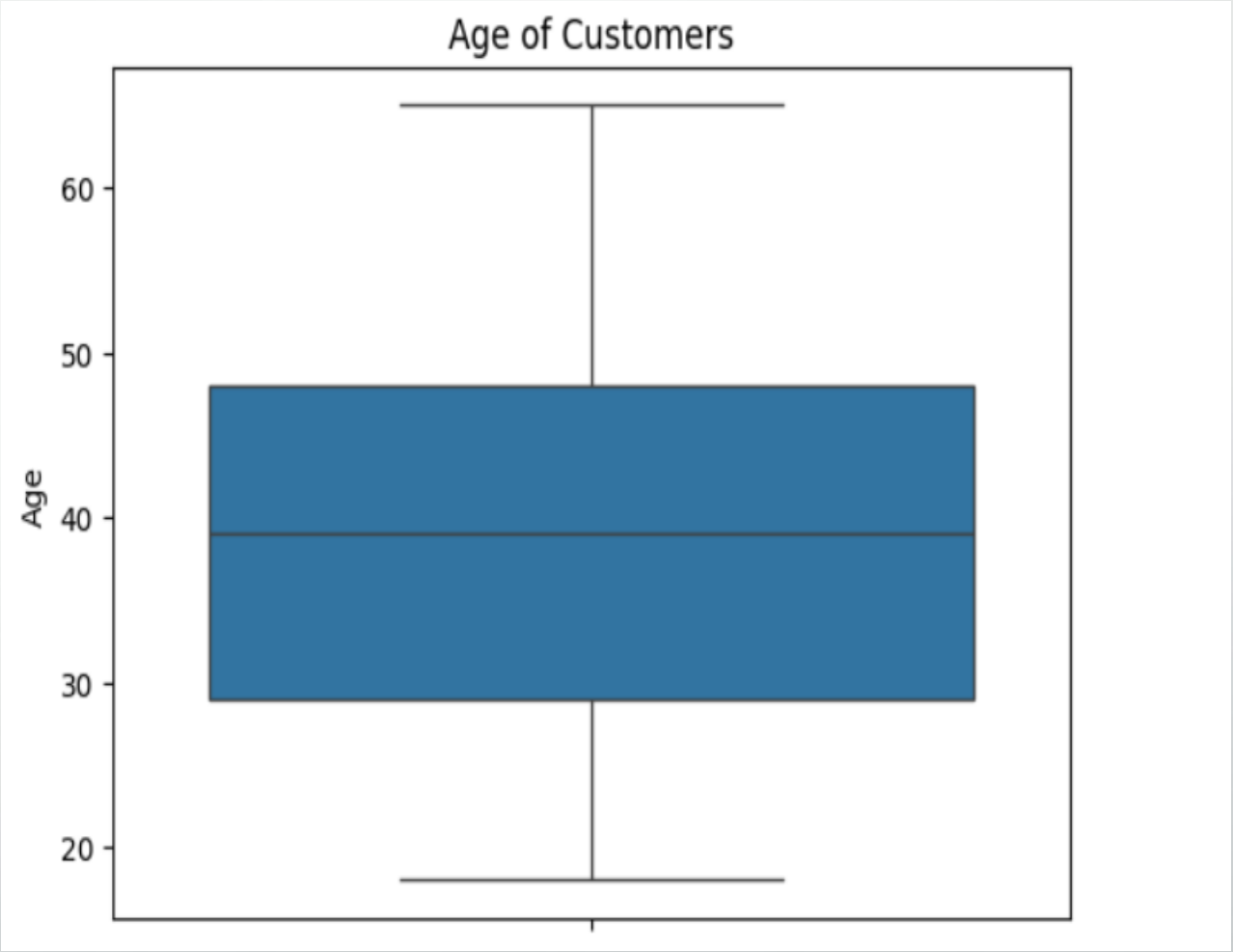
```
df.isna().sum()
```

```
CustomerID           0
Age                  0
Gender               0
Tenure               0
Usage Frequency      0
Support Calls        0
Payment Delay        0
Subscription Type    0
Contract Length      0
Total Spend          0
Last Interaction     0
Churn                0
dtype: int64
```

Visualization

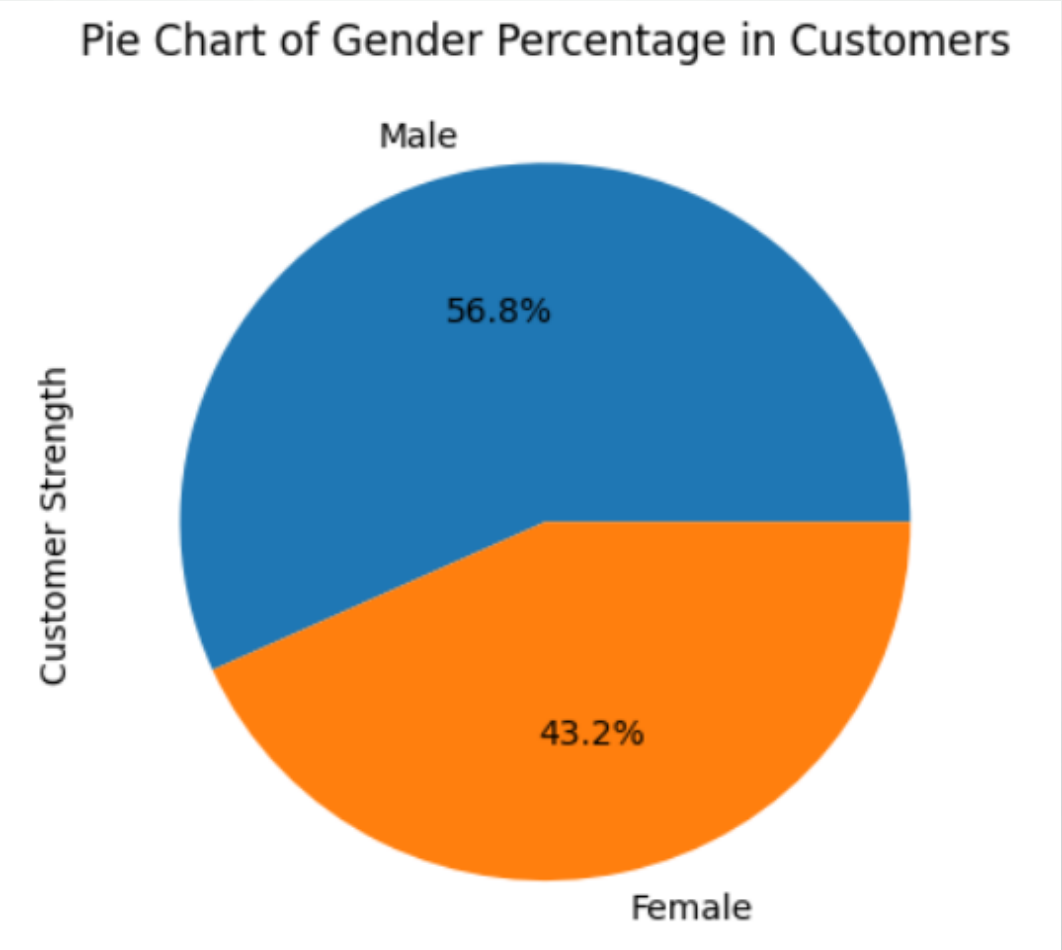
Age of customers

- We can see the customers ages are around 19-65 years.
- The maximum usage of customers is around 29-49 years.



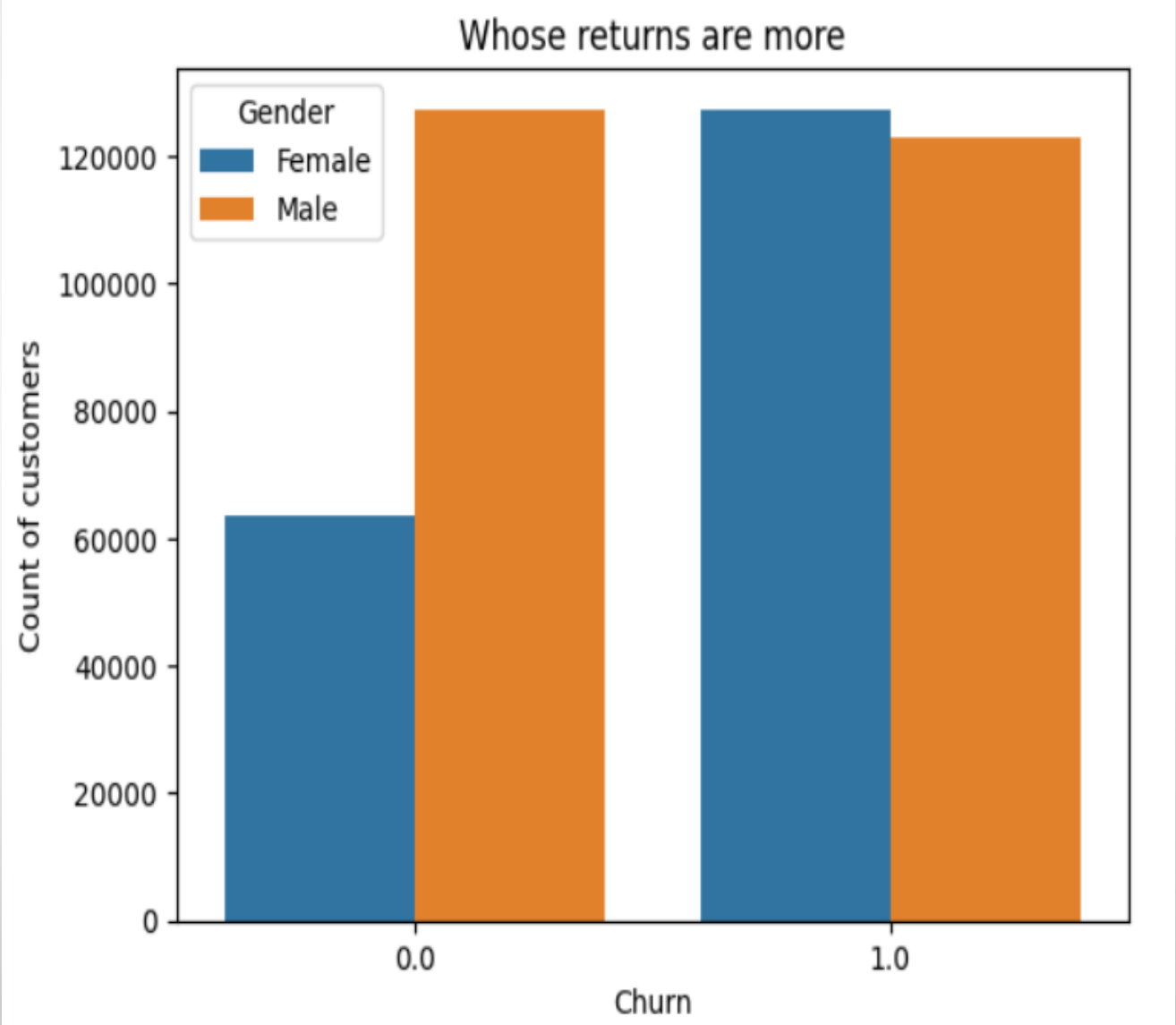
Gender of customers

- The male customers are then the female customers.
- The male customers are 56.8% and female customers are 43.2%.



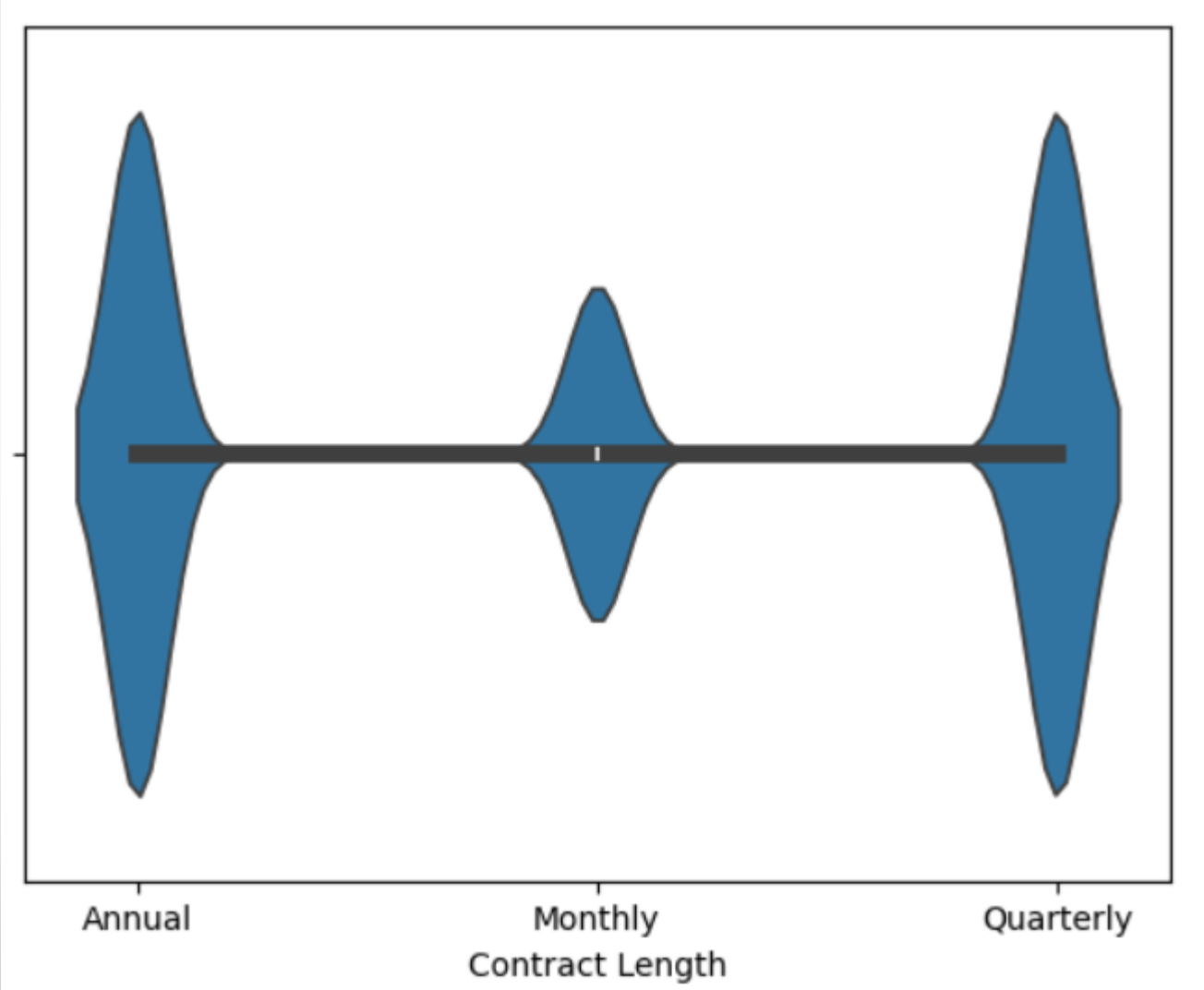
Churn of customers with Gender

- Male customers are have churn are more then the customers leaving.
- Female customers are leaving more then the customers who are staying.



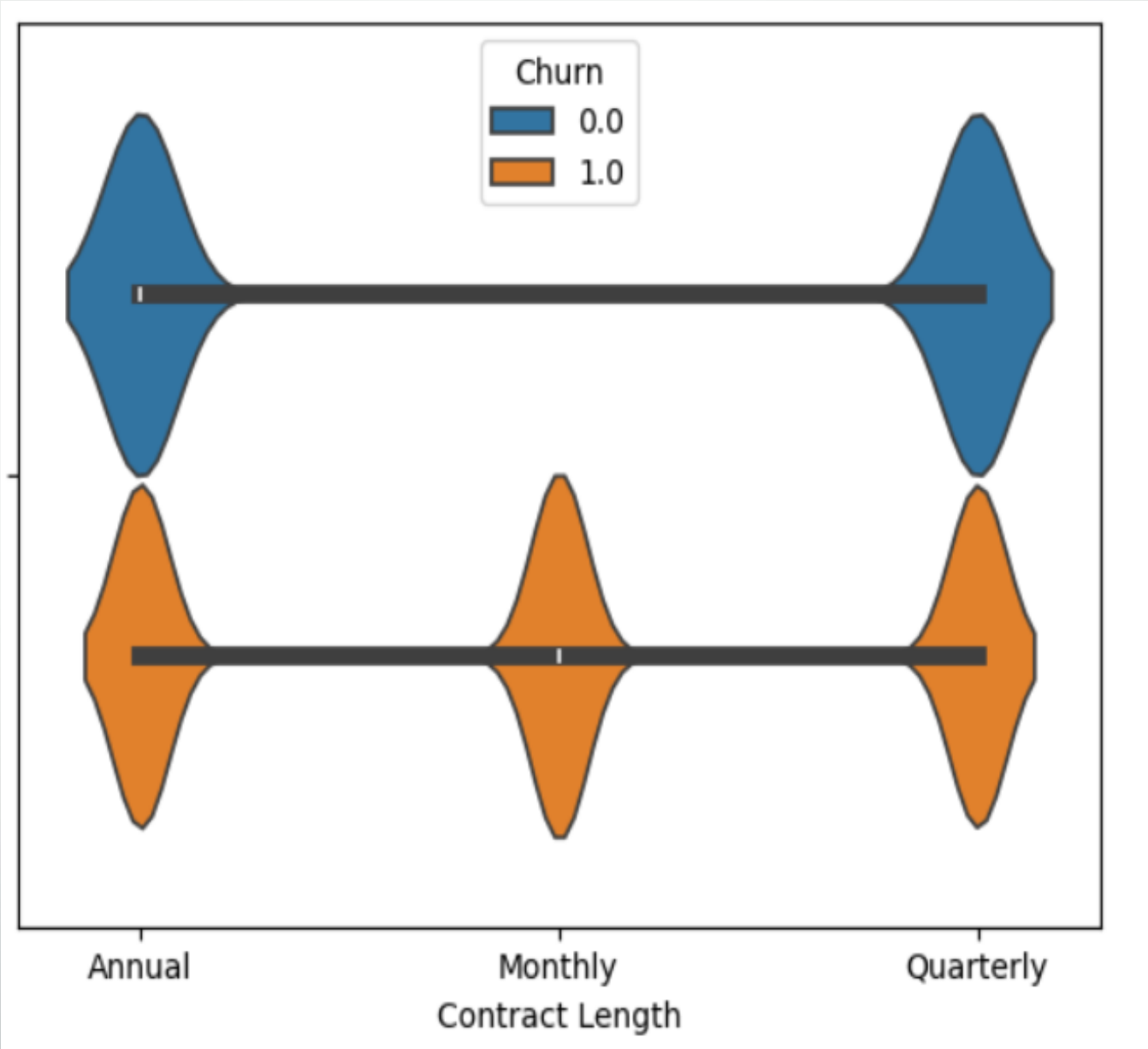
Contract Length

- Contract lenths are three types. They are Annual, Monthly, Quarterly.
- Among the three Contract types the monthly Contract is taken very less compared to other two contracts.



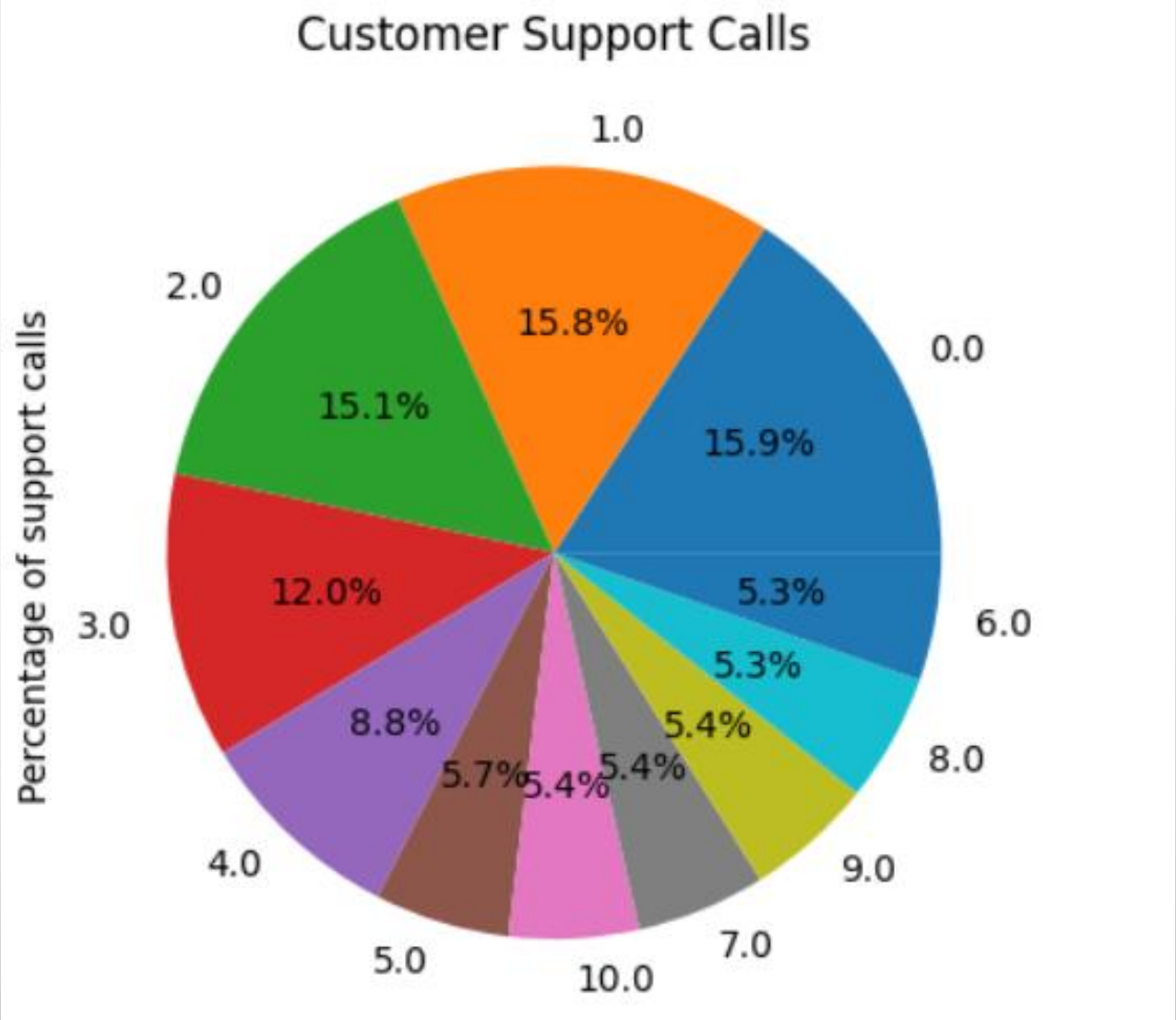
Churn Based Contract Length

- Monthly contracts have No Churn.
- Customers are leaving in both the annual and quarterly contracts.



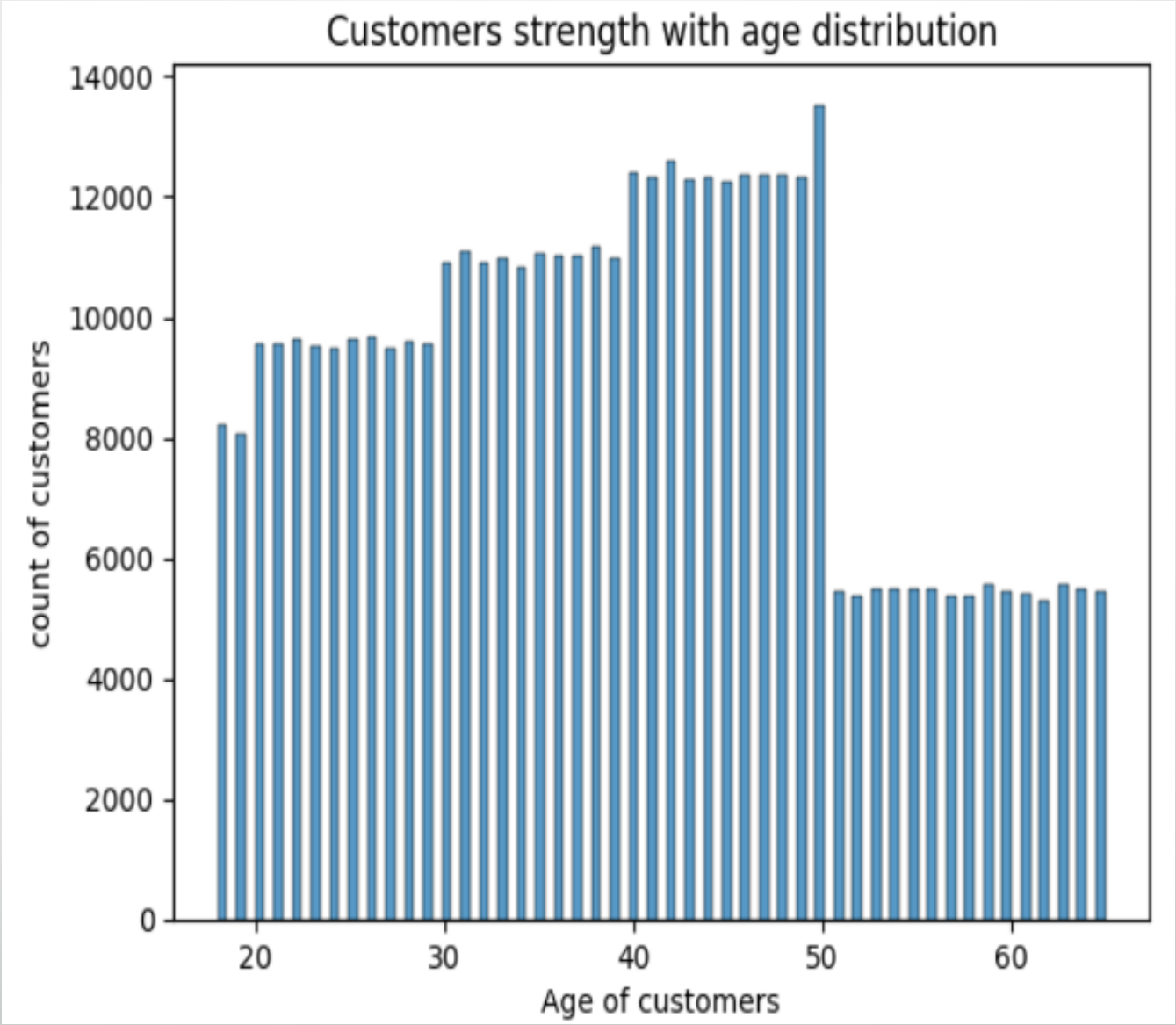
Customer Support Calls

- Most of the customers are making support calls.
- Mostly of the customers problems are solved in 1-3 calls others are taking 4-8 calls.



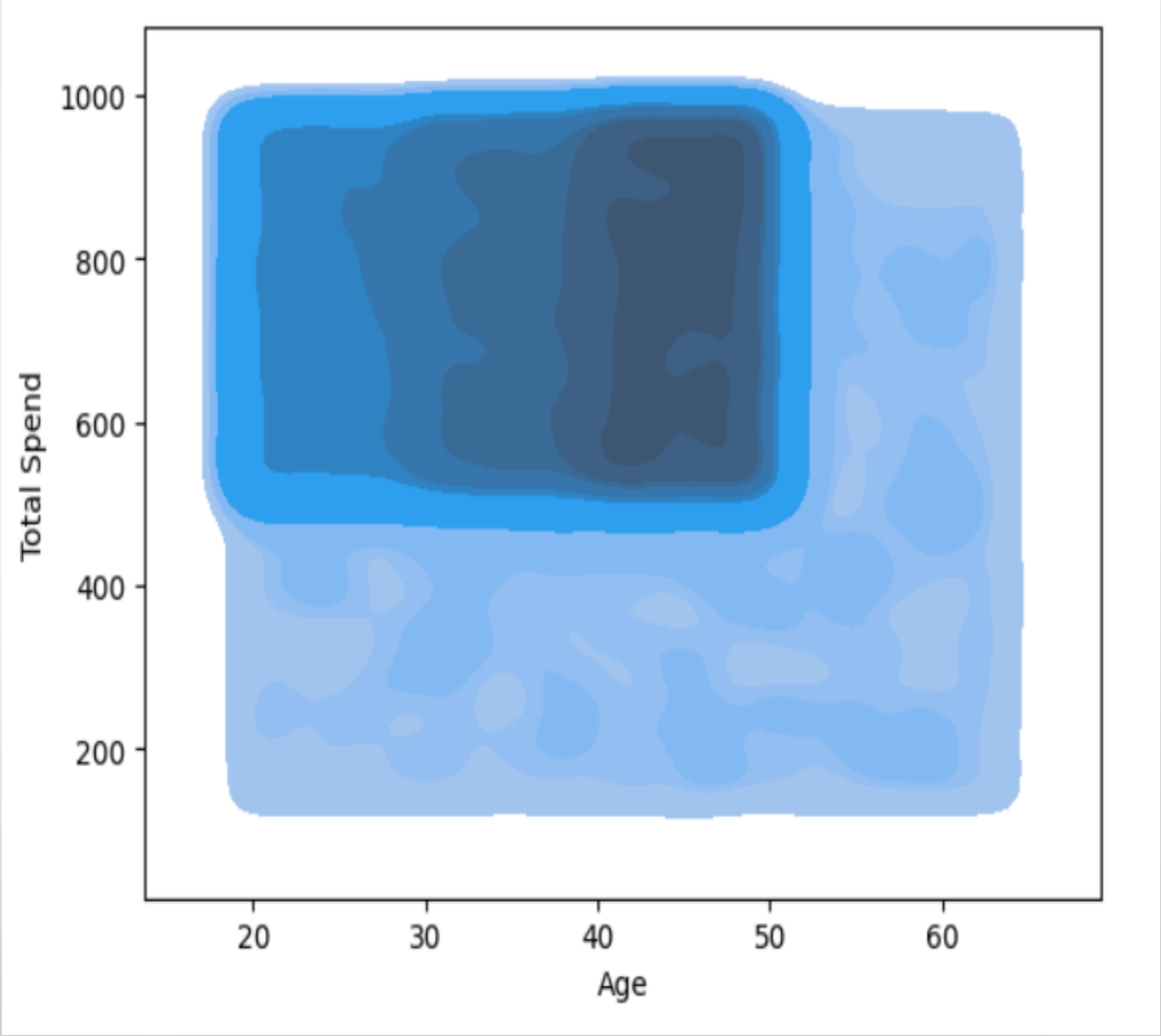
Customer Age Distribution

- Customers with 50+ age are very less.
- Customers age 30-50 are very high exceeding 10000.



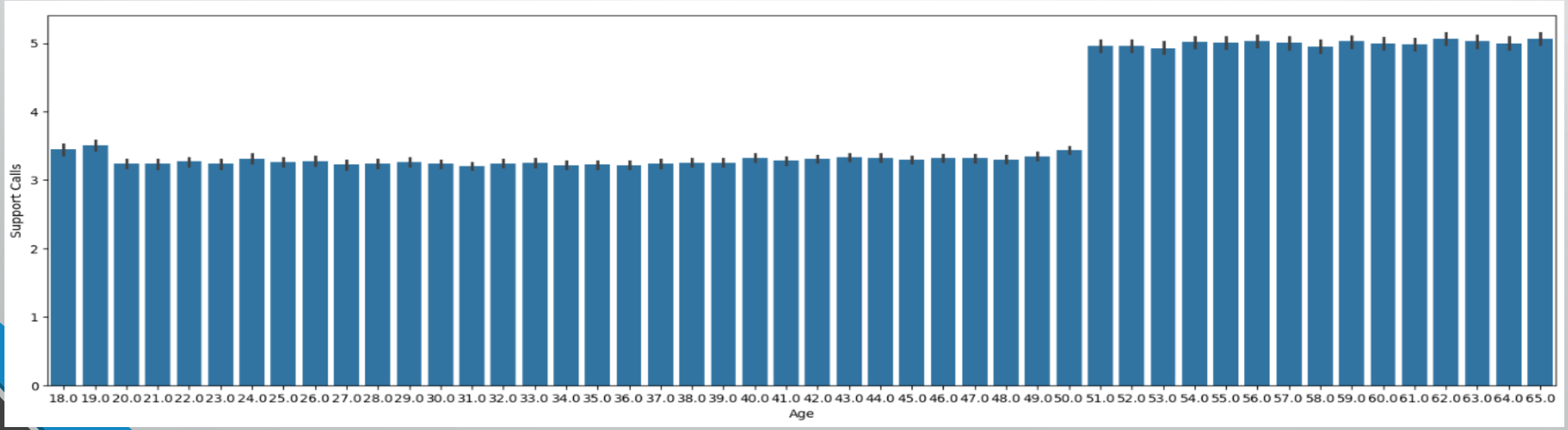
Age and Money Spending

- Customers with age 40 – 50 have high money spending.
- Customers with 50+ age are spending less.



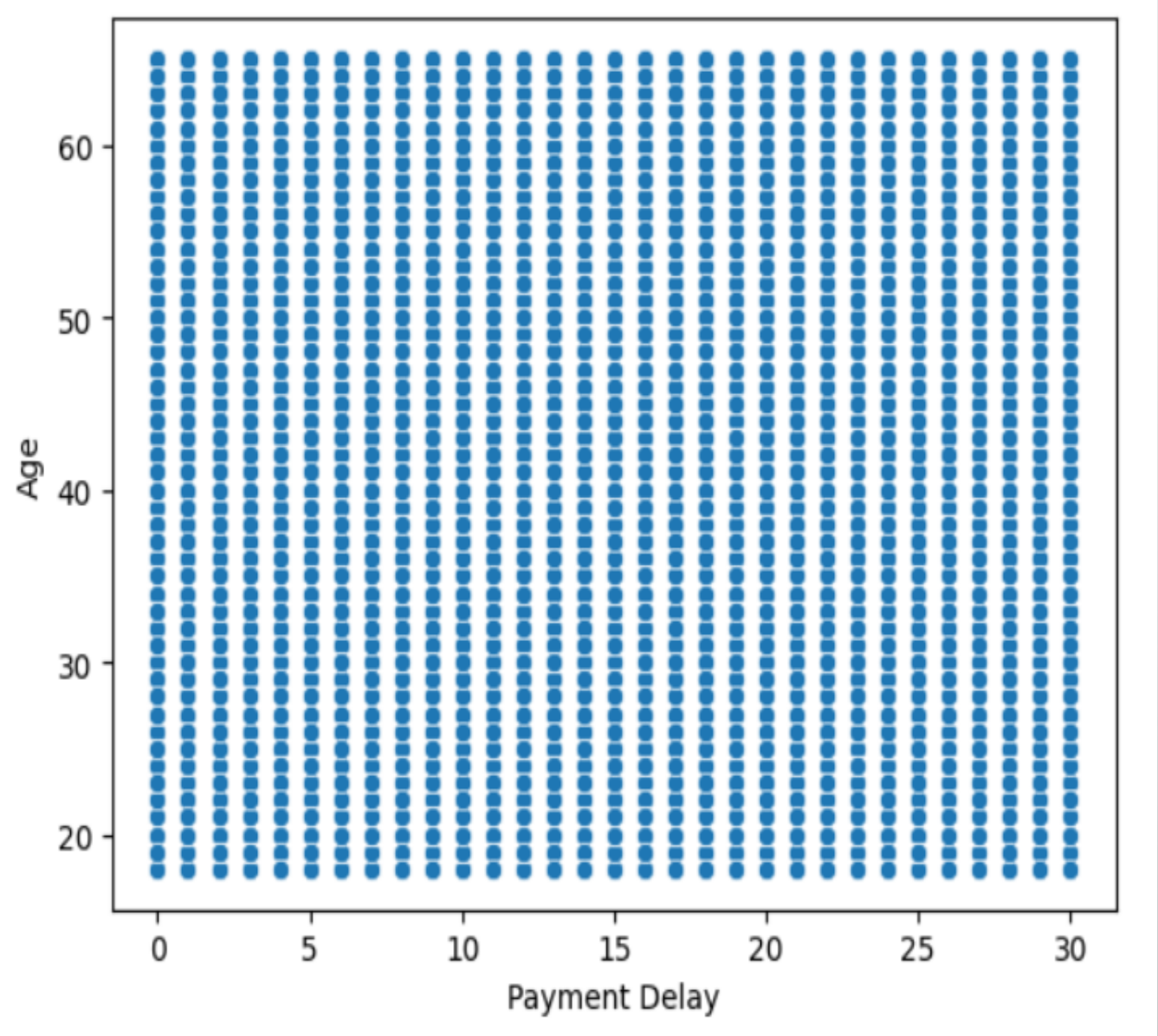
Age and Suppot Calls

- All the age people calls for support but people age with more then 50 are making more calls.



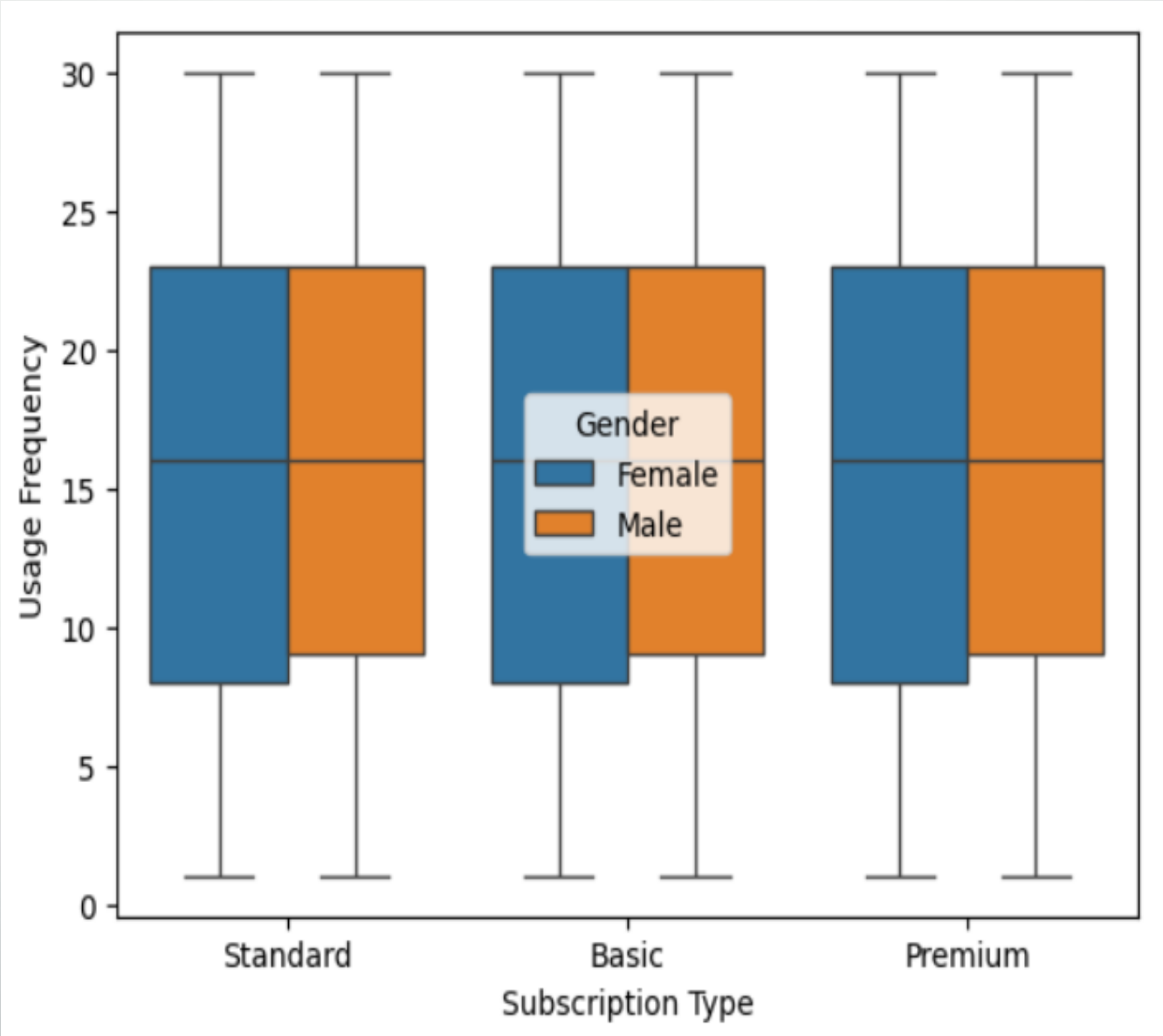
Payment Delay

- All the age people are delaying the payments regardless of age.
- Payment delays are done 0 to 30 times.



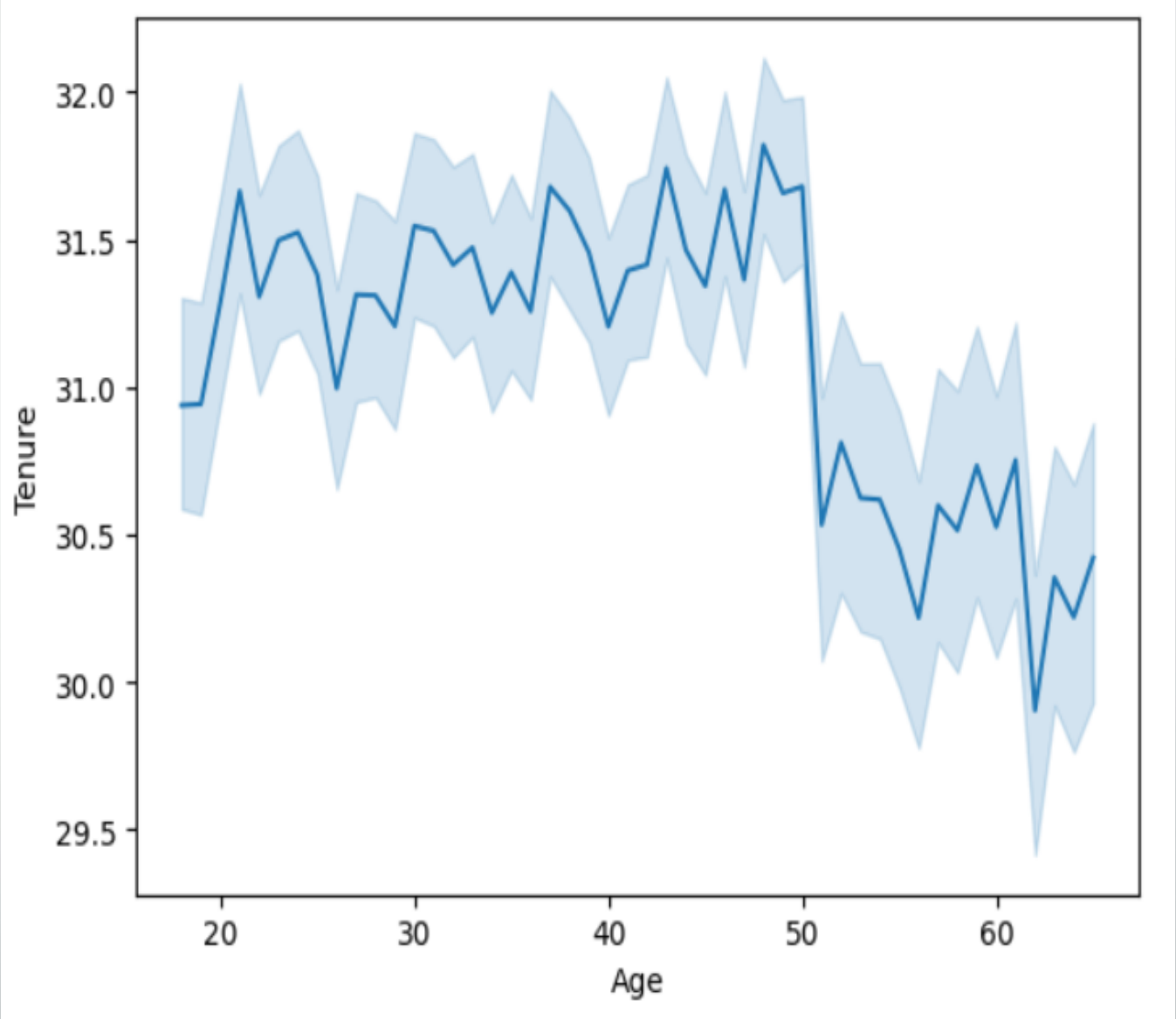
Gender and Subscription and Usage frequency

- All the subscriptions have almost same usage frequency.
- Female customers have 8 – 24 usage frequency and male customers have 9 – 24 usage frequency.



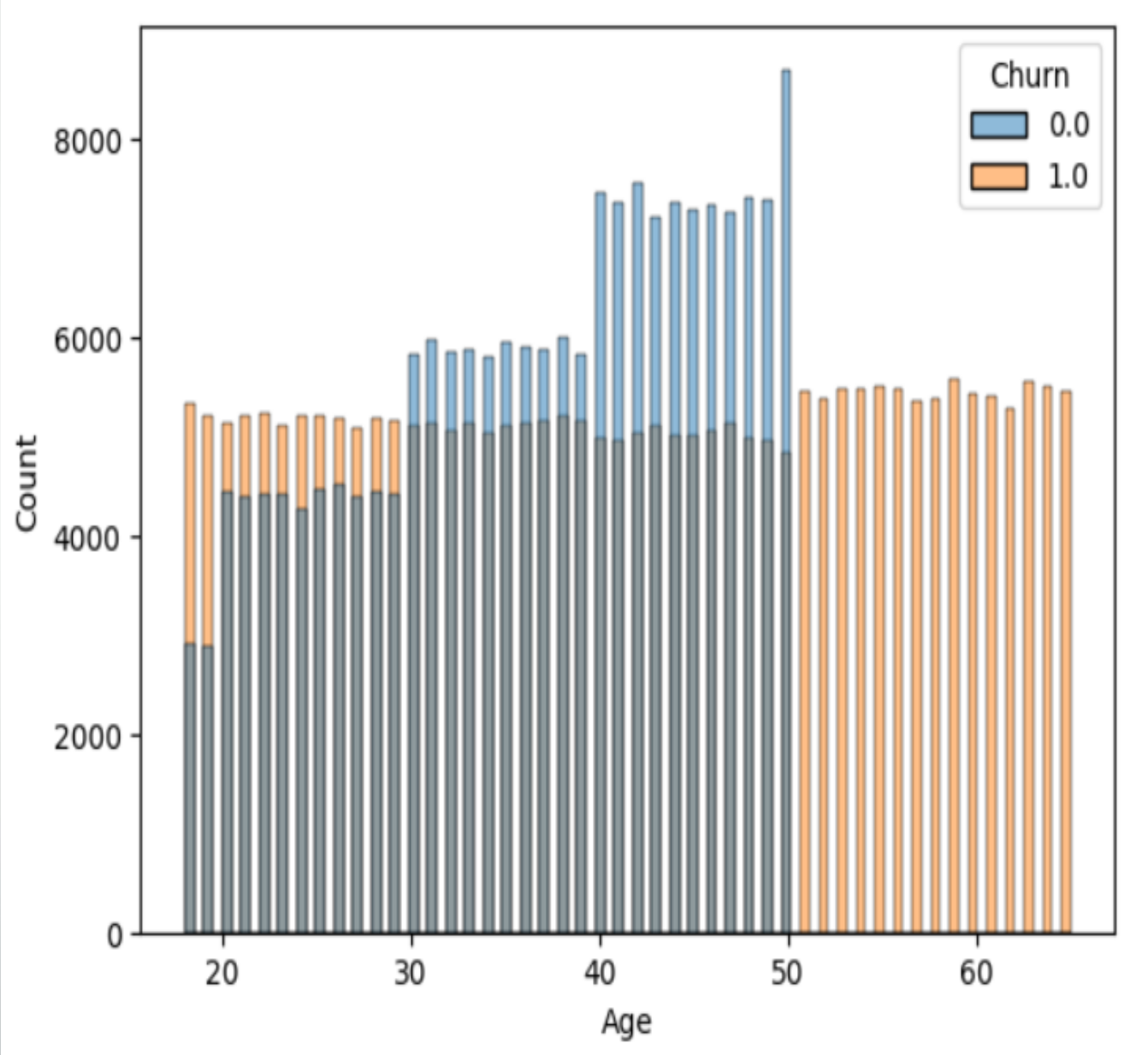
Age and Tenure

- Age of 20 – 50 having high tenure.
- But after 50 there is low tenure.

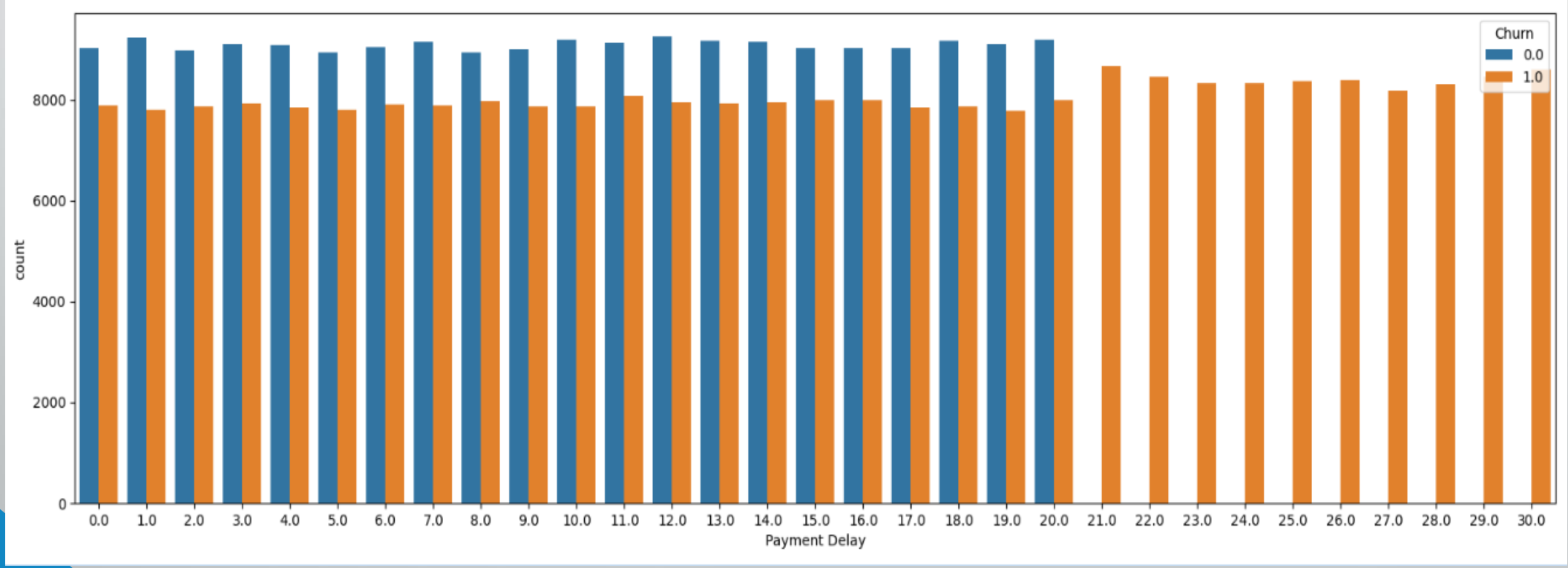


Age and Churn

- Age of 50+ people are having no churn.
- Age of 30 – 50 have high churn.

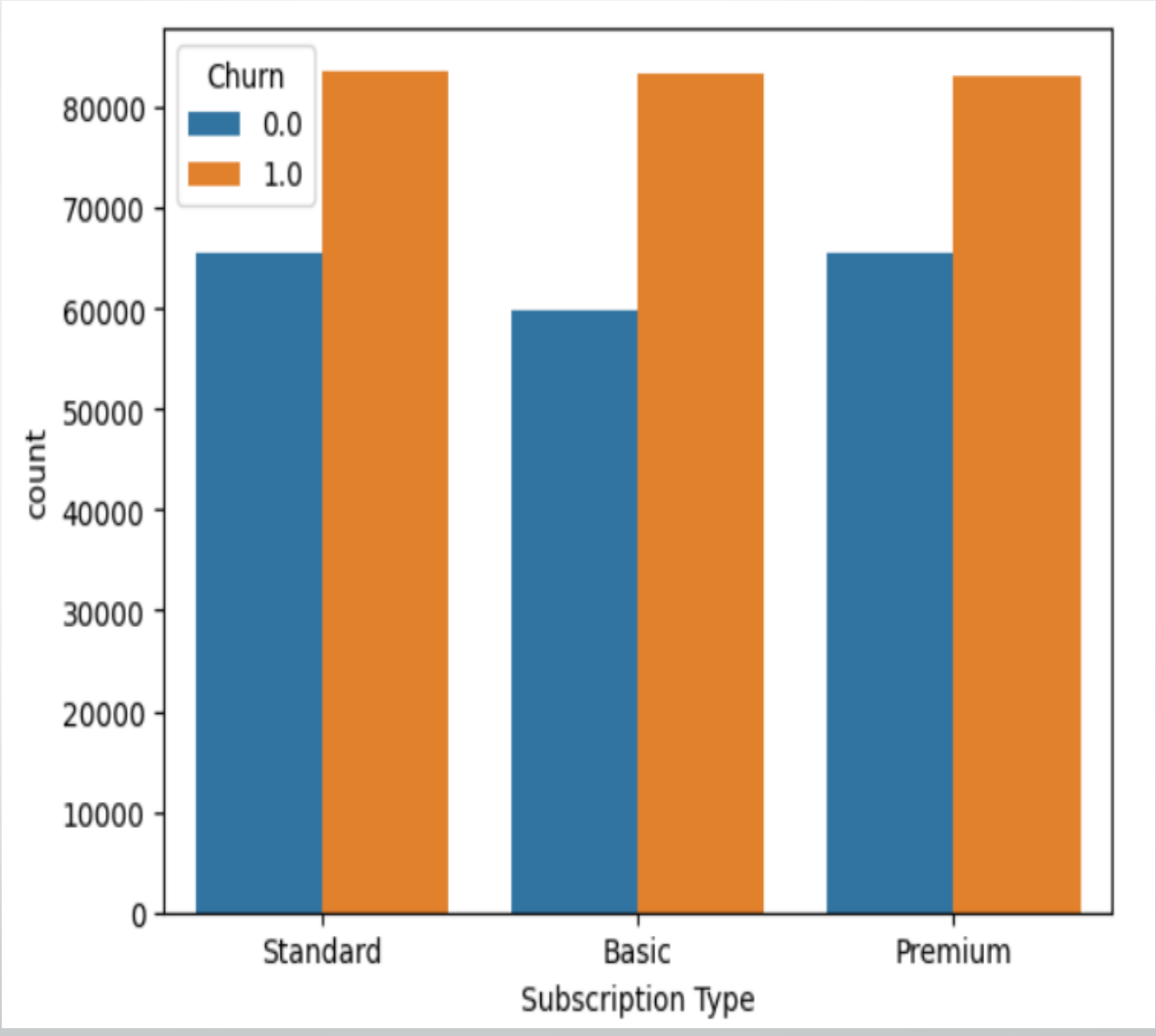


Payment Delay



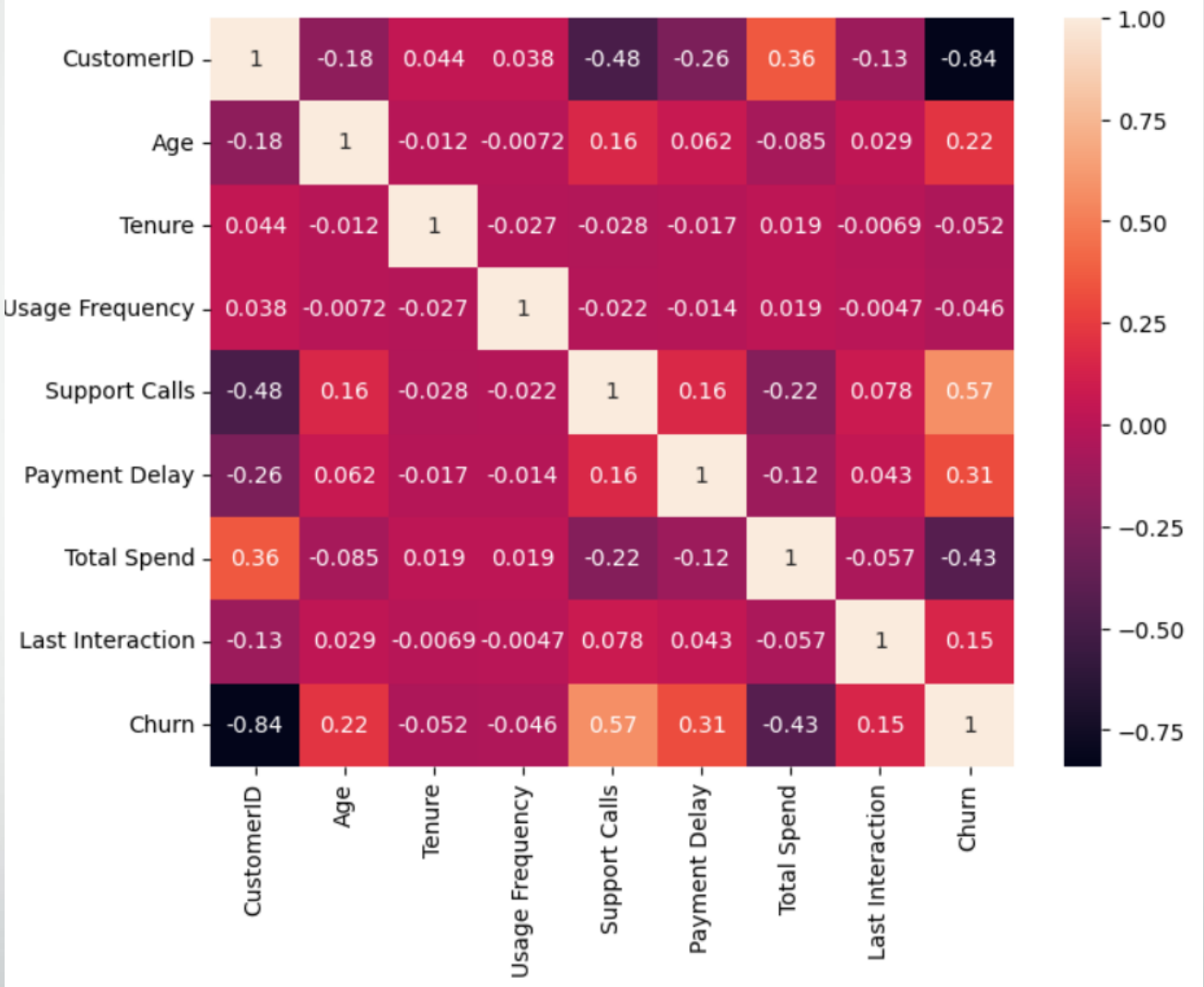
Subscription Types

- There are three types of subscriptions. They are Standard, Basic, Premium.
- In all the subscriptions customers leaving are same but in the basic subscription churn is little low.



HeatMap

- You can see correlation of every columns and rows in the heatmap



Modeling

Scikit Learn

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
```

- **Scikit-learn** (also written as **sklearn**) is an open-source **machine learning library in Python** used for building and applying machine learning models.
- We Import the scikit learn libraries like train_test_split and minmaxscaler.
- Train_test_split library used for training and testing the data.
- MinMaxScaler used for feature scaling.

Encoding

```
df=pd.get_dummies(df)
```

```
x=df.drop(columns='Churn')
y=df[['Churn']]
```

- We are using the one_hot coding method to convert each categorical columns into numerical columns
- Next splitting the data into the x and y variables

x.head

- **x** is containing of data where no categorical columns.
- It does not have the target column also.

```
x.head()
```

e y	Support Calls	Payment Delay	Total Spend	Last Interaction	Gender_Female	Gender_Male	Subscription Type_Basic	Subscription Type_Premium	Subscription Type_Standard	Contract Length_Annual	Contract Length_Monthly	Contract Length_Quarterly
0	5.0	18.0	932.0	17.0	True	False	False	False	True	True	False	False
0	10.0	8.0	557.0	6.0	True	False	True	False	False	False	True	False
0	6.0	18.0	185.0	3.0	True	False	True	False	False	False	False	True
0	7.0	7.0	396.0	29.0	False	True	False	False	True	False	True	False
0	5.0	8.0	617.0	20.0	False	True	True	False	False	False	True	False

y.head

- y is containing of only Churn column which is the target column.
- It has the values of 0 and 1 .

```
y.head()
```

	Churn
0	1.0
1	1.0
2	1.0
3	1.0
4	1.0

Train and Test

```
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=100)
```

- Here we perform the training and testing of the data.
- The test data will be taken into 20% then the train data will 80%.
- The train data will be divided into x_train and y_train.
- The test data will be divided into x_test and y_test.

Dimentions of data

- After splitting the data 80% of the data went into train data.
- And 20% of the data went into test data.
- As shown in figure train data containing of 352665 rows. And in test data it contains 88167 rows.

```
x_train.shape
```

```
(352665, 16)
```

```
x_test.shape
```

```
(88167, 16)
```

```
y_train.shape
```

```
(352665, 1)
```

```
y_test.shape
```

```
(88167, 1)
```

x_train

```
x_train
```

e y	Support Calls	Payment Delay	Total Spend	Last Interaction	Gender_Female	Gender_Male	Subscription Type_Basic	Subscription Type_Premium	Subscription Type_Standard	Contract Length_Annual	Contract Length_Monthly	Contract Length_Quarterly
)	4.0	23.0	885.00	1.0	False	True	False	True	False	True	False	False
)	3.0	19.0	604.00	20.0	False	True	False	True	False	False	True	False
)	0.0	3.0	557.64	16.0	False	True	True	False	False	False	False	True
)	2.0	20.0	373.00	10.0	False	True	False	True	False	True	False	False
)	2.0	9.0	912.82	15.0	True	False	False	True	False	True	False	False
..	...	...	...	...	...	...	...	...	...	...	...	...
)	3.0	17.0	855.84	7.0	True	False	True	False	False	False	False	True
)	1.0	14.0	520.82	1.0	False	True	False	False	True	True	False	False
)	1.0	18.0	224.50	19.0	True	False	False	False	True	False	False	True
)	5.0	14.0	244.00	4.0	True	False	True	False	False	False	False	True
)	2.0	5.0	870.21	3.0	False	True	True	False	False	False	False	True

x_test

```
x_test
```

e y	Support Calls	Payment Delay	Total Spend	Last Interaction	Gender_Female	Gender_Male	Subscription Type_Basic	Subscription Type_Premium	Subscription Type_Standard	Contract Length_Annual	Contract Length_Monthly	Contract Length_Quarterly
)	6.0	3.0	977.00	20.0	True	False	False	False	True	False	False	True
)	7.0	19.0	780.79	3.0	True	False	True	False	False	False	False	True
)	9.0	30.0	313.00	20.0	True	False	False	False	True	False	True	False
)	1.0	3.0	400.00	8.0	True	False	False	False	True	False	False	True
)	7.0	10.0	324.00	16.0	False	True	False	False	True	True	False	False
..	...	...	...	...	...	...	...	...	...	...	...	...
)	0.0	19.0	666.68	7.0	True	False	False	True	False	False	False	True
)	10.0	1.0	463.00	28.0	True	False	True	False	False	True	False	False
)	0.0	23.0	667.95	18.0	True	False	True	False	False	False	False	True
)	1.0	9.0	312.00	8.0	True	False	False	True	False	False	True	False
)	4.0	24.0	287.00	26.0	True	False	True	False	False	False	True	False

y_train

```
y_train
```

	Churn
170896	1.0
56774	1.0
320768	0.0
144694	1.0
309934	0.0
...	...
339800	0.0
253800	0.0
210756	1.0
56088	1.0
300553	0.0

352665 rows × 1 columns

y_test

```
y_test
```

	Churn
146546	1.0
247393	1.0
85067	1.0
169486	1.0
127462	1.0
...	...
292366	0.0
5452	1.0
237520	1.0
141456	1.0
185131	1.0

88167 rows × 1 columns

Feature Scaling

```
scaler=MinMaxScaler()
x_train_scaled=scaler.fit_transform(x_train)
x_test_scaled=scaler.fit_transform(x_test)
```

- We are using the MinMaxScaler for feature scaling.
- In this step we are transforming the data.

Scaled Data

```
print(x_train_scaled)
```

```
[[0.39045816 0.9787234  0.45762712 ... 1.         0.         0.        ]
 [0.12856752 0.12765957 0.16949153 ... 0.         1.         0.        ]
 [0.7286293  0.21276596 0.54237288 ... 0.         0.         1.        ]
 ...
 [0.4800632  0.91489362 0.23728814 ... 0.         0.         1.        ]
 [0.12697196 0.87234043 0.06779661 ... 0.         0.         1.        ]
 [0.68370456 0.06382979 0.27118644 ... 0.         0.         1.        ]]
```

```
print(x_test_scaled)
```

```
[[0.33633125 0.59574468 0.28813559 ... 0.         0.         1.        ]
 [0.56513082 0.4893617  0.49152542 ... 0.         0.         1.        ]
 [0.19414017 0.08510638 0.47457627 ... 0.         1.         0.        ]
 ...
 [0.5421453  0.65957447 0.76271186 ... 0.         0.         1.        ]
 [0.32492405 0.87234043 0.30508475 ... 0.         1.         0.        ]
 [0.4220797  0.12765957 0.57627119 ... 0.         1.         0.        ]]
```

Logistic Regression

Scikit Libraries

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score,confusion_matrix,classification_report
```

- We are importing the scikit libraries of LogisticRegression to perform Logistic Regression.
- And to perform the metrics operations we are using accuracy_score, confusion_matrix, classification_report Libraries.

Model Building

- Here we are Building the Model of Logistic Regression.

```
model=LogisticRegression()
model.fit(x_train_scaled,y_train)
```

```
▾ LogisticRegression
LogisticRegression()
```

Output

- We can see the outcome of model in the picture.
- The accuracy of the model is 98.04%.
- The confusion matrix and classification report as follows.

```
pred=model.predict(x_test_scaled)
```

```
print('Accuracy_score:',accuracy_score(y_test,pred))
print('confusion matrix:\n',confusion_matrix(y_test,pred))
print('classification_report:\n',classification_report(y_test,pred))
```

```
Accuracy_score: 0.9804461986911203
confusion matrix:
 [[37302   784]
 [  940 49141]]
classification_report:
               precision    recall  f1-score   support

         0.0       0.98      0.98      0.98     38086
         1.0       0.98      0.98      0.98     50081

    accuracy                           0.98     88167
   macro avg       0.98      0.98      0.98     88167
weighted avg       0.98      0.98      0.98     88167
```

Decision Tree

Model Building

- Here we first import the scikit library of decision tree.
- And build the model for Decision Tree Classifier.

```
from sklearn.tree import DecisionTreeClassifier
```

```
#Decision Tree
model=DecisionTreeClassifier(random_state=42)
model.fit(x_train,y_train)
```

```
DecisionTreeClassifier
DecisionTreeClassifier(random_state=42)
```

Accuracy

```
y_pred=model.predict(x_test)
print('Decision Tree Accuracy:',accuracy_score(y_test,y_pred))
```

```
Decision Tree Accuracy: 0.9997958419816938
```

- The accuracy of decision tree is 99.9%.

Random Forest

Model Building

- Here we first import the scikit library of Random Forest Classifier.
- Now we build a model for the Random Forest Classifier.

```
from sklearn.ensemble import RandomForestClassifier
```

```
model=RandomForestClassifier(n_estimators=100,max_depth=None,random_state=42)
```

```
model.fit(x_train,y_train)
```

```
RandomForestClassifier
RandomForestClassifier(random_state=42)
```

Accuracy

```
y_pred=model.predict(x_test)
print('Random Forest Accuracy:',accuracy_score(y_test,y_pred))
```

```
Random Forest Accuracy: 0.9997051050846689
```

- The accuracy of the random forest classifier is 99.9%.